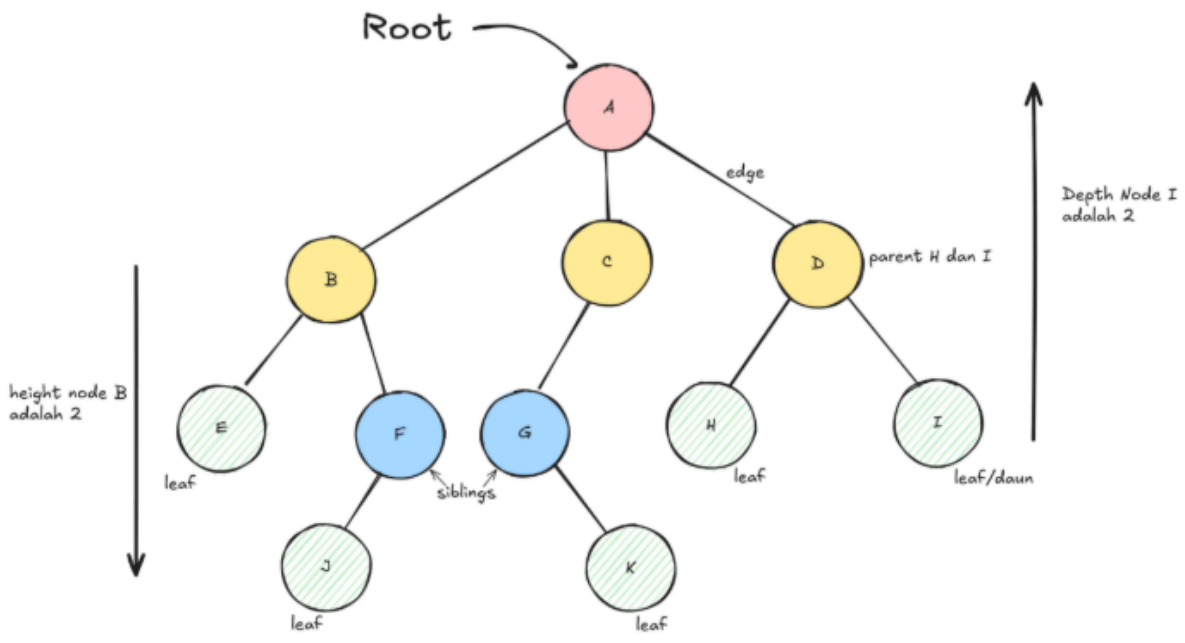
TREE

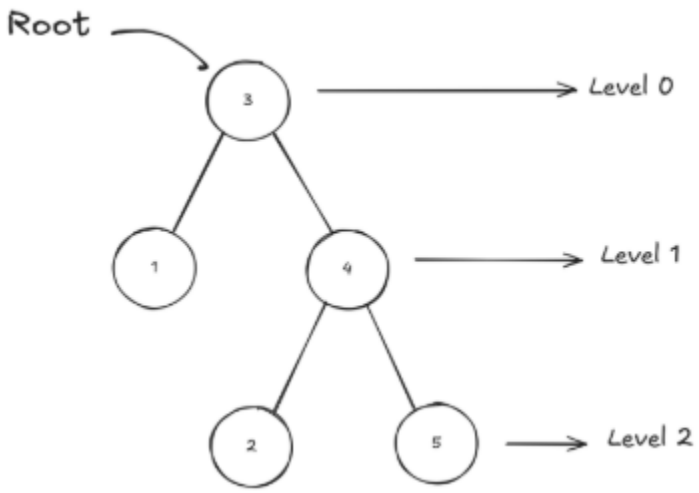
1. Pengantar Tree

Tree adalah struktur data yang menyerupai linked list, namun tidak seperti linked list di mana setiap node menunjuk ke node berikutnya secara linear, dalam Tree, setiap node dapat menunjuk ke beberapa node lain. Tree adalah contoh dari struktur data non-linear. Struktur Tree digunakan untuk merepresentasikan sifat hierarkis dari suatu data dalam bentuk grafis. Pada ADT (Abstract Data Type) Tree, urutan elemen tidaklah penting. Jika kita membutuhkan informasi pengurutan, struktur data linear seperti linked list, stack, atau queue lebih cocok digunakan. Struktur data hierarkis non-linear yang terdiri dari node yang terhubung melalui edge. Untuk struktur data linear, kompleksitas waktu meningkat seiring dengan ukuran data. Sedangkan Tree memungkinkan akses yang lebih cepat dan mudah.



- Node: Entitas yang menyimpan data (kunci atau nilai) dan memiliki penunjuk (pointer) yang mengarah ke node lain.
- Edge: Penghubung yang menunjukkan hubungan dari parent ke child.

- Adjacency (Ketetanggaan): Dua node disebut adjacent (bertetangga) jika ada edge langsung di antara keduanya.
- Root (Akar/Awal/Sumber): Node yang tidak memiliki parent. Dalam sebuah Tree hanya ada satu node root.
- Parent: Node yang memiliki satu atau lebih node anak (child).
- Child: Node yang memiliki parent.
- Leaf Node (Node Daun): Node yang tidak memiliki child sama sekali.
- Internal Node: Node yang memiliki satu atau lebih node anak.
- Sibling (Saudara kandung): Node-node yang memiliki parent yang sama.
- Predecessor (Pendahulu): Node yang berada diatas node tertentu.
- Successor (Penerus) & Subtree: Node yang berada dibawah node tertentu.
- Ancestor (Leluhur) & Descendant (Keturunan): Node P adalah ancestor dari node jika ada jalur dari root menuju P dan Q berada di jalur tersebut. Sebaliknya, node Q disebut descendant dari P.
- Derajat Node (Degree of a Node): Jumlah anak yang dimiliki oleh node tersebut.
- Level (Tingkat): Kumpulan semua node yang berada pada kedalaman (depth) yang sama. Node root berada di level nol.



- Depth (Kedalaman): Panjang jalur (jumlah edge) dari root ke node tertentu.

- Height of a Node (Tinggi sebuah node): Panjang jalur dari node tersebut ke node terdalam.
- Height of a Tree (Tinggi dari Tree): Tinggi maksimum di antara semua node dalam Tree dan kedalaman dari Tree adalah kedalaman maksimum di antara semua node dalam Tree.
- Forest (Hutan): Kumpulan Tree yang saling terpisah. Sebuah forest terbentuk ketika kita memotong root dari sebuah Tree.

Lebih lanjutnya, silahkan pelajari source code berikut ini.

```
#include <iostream>
#include <queue>
using namespace std;

// Struktur Node untuk Binary Tree
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};

// Operasi Binary Search Tree (BST)

// Fungsi insert
Node* insert(Node* root, int val) {
    if (root == nullptr) {
        return new Node(val);
    }

    if (val < root->data) {
        root->left = insert(root->left, val);
```

```
    } else if (val > root->data) {
        root->right = insert(root->right, val);
    }
    return root;
}

// Fungsi search
Node* search(Node* root, int val) {
    if (root == nullptr || root->data == val) {
        return root;
    }

    if (val < root->data) {
        return search(root->left, val);
    }
    return search(root->right, val);
}

// Cari node dengan nilai minimum
Node* findMinValueNode(Node* node) {
    Node* current = node;
    while (current != nullptr && current->left != nullptr) {
        current = current->left;
    }
    return current;
}

// Fungsi delete
Node* deleteNode(Node* root, int val) {
    if (root == nullptr) {
        return root;
    }

    if (val < root->data) {
        root->left = deleteNode(root->left, val);
    } else if (val > root->data) {
        root->right = deleteNode(root->right, val);
    } else {
        // Node dengan satu atau nol anak
```

```
        if (root->left == nullptr) {
            Node* temp = root->right;
            delete root;
            return temp;
        } else if (root->right == nullptr) {
            Node* temp = root->left;
            delete root;
            return temp;
        }

        // Node dengan dua anak
        Node* temp = findMinValueNode(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

// Traversal In-Order
void inorderTraversal(Node* root) {
    if (root == nullptr) return;

    inorderTraversal(root->left);
    cout << root->data << " ";
    inorderTraversal(root->right);
}

// Traversal Pre-Order
void preorderTraversal(Node* root) {
    if (root == nullptr) return;

    cout << root->data << " ";
    preorderTraversal(root->left);
    preorderTraversal(root->right);
}

// Traversal Post-Order
void postorderTraversal(Node* root) {
    if (root == nullptr) return;
```

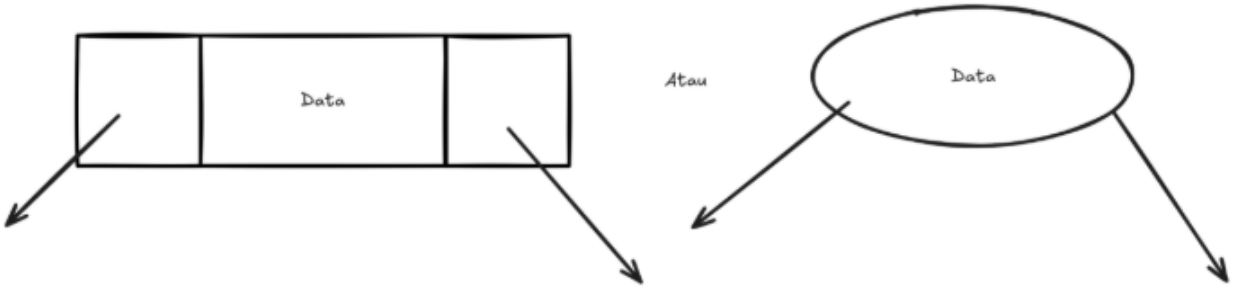
```
    postorderTraversal(root->left);
    postorderTraversal(root->right);
    cout << root->data << " ";
}

// Traversal Level-Order (BFS)
void bfs(Node* root) {
    if (root == nullptr) return;

    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        Node* current = q.front();
        q.pop();

        cout << current->data << " ";

        if (current->left != nullptr)
            q.push(current->left);
        if (current->right != nullptr)
            q.push(current->right);
    }
}

// Fungsi utama
int main() {
    Node* root = nullptr;

    cout << "Membangun BST dengan nilai: 50, 30, 70, 20, 40, 60, 80\n";

    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);
```

```
    cout << "\nIn-Order   : ";
    inorderTraversal(root);
    cout << "\nPre-Order  : ";
    preorderTraversal(root);
    cout << "\nPost-Order : ";
    postorderTraversal(root);
    cout << "\nLevel-Order: ";
    bfs(root);

    cout << "\n\nPencarian nilai 40: ";
    cout << (search(root, 40) != nullptr ? "Ditemukan" : "Tidak
ditemukan");

    cout << "\nPencarian nilai 99: ";
    cout << (search(root, 99) != nullptr ? "Ditemukan" : "Tidak
ditemukan");

    cout << "\n\nHapus node 20\n";
    root = deleteNode(root, 20);
    inorderTraversal(root);

    cout << "\n\nHapus node 30\n";
    root = deleteNode(root, 30);
    inorderTraversal(root);

    cout << "\n\nHapus node 50\n";
    root = deleteNode(root, 50);
    inorderTraversal(root);

    return 0;
}
```

2. Binary Tree

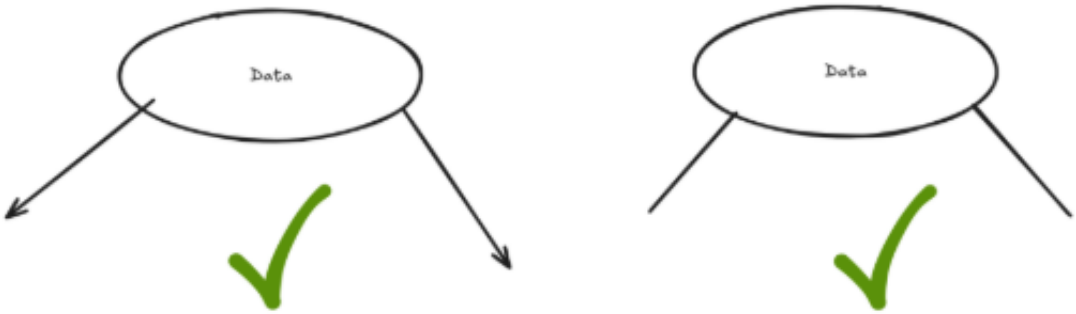
Sekarang, mari kita definisikan struktur dari binary Tree. Untuk mempermudah, asumsikan data yang disimpan di node adalah bilangan bulat. Salah satu cara untuk

merepresentasikan sebuah node adalah dengan memiliki data dan dua link, yang menunjuk ke anak kiri dan anak kanan.




```
struct BinaryTreeNode {
    int data;
    struct BinaryTreeNode *left;
    struct BinaryTreeNode *right;
};
```

Catatan: Dalam Tree, arah standar mengalir dari parent ke anak, jadi tidak wajib menunjukkan arah panah. Kita asumsikan kedua representasi di atas memiliki arti yang sama.



Tree traversal

Untuk memahami sebuah Tree, kita perlu melakukan Tree traversal, yaitu proses mengunjungi setiap node di dalamnya. Metode penjelajahan ini sangat penting karena

mempertimbangkan struktur hirarkis dari Tree. Pada dasarnya, setiap Tree terdiri dari satu node yang menyimpan data, serta dua subTree (subTree kiri dan subTree kanan).

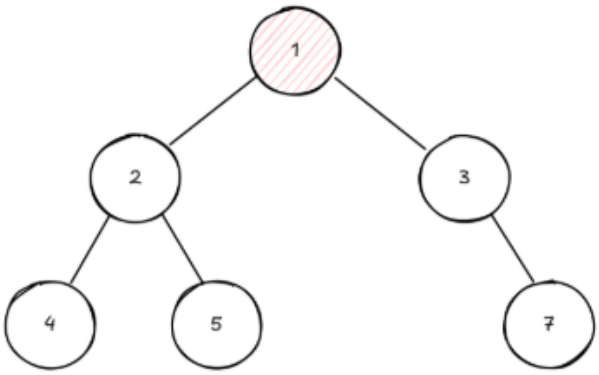
Tipe-tipe traversal

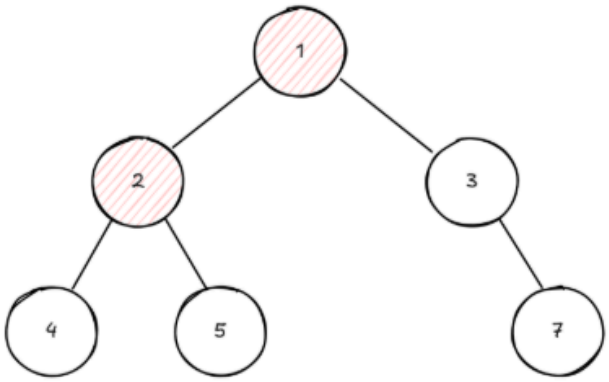
In order traversal

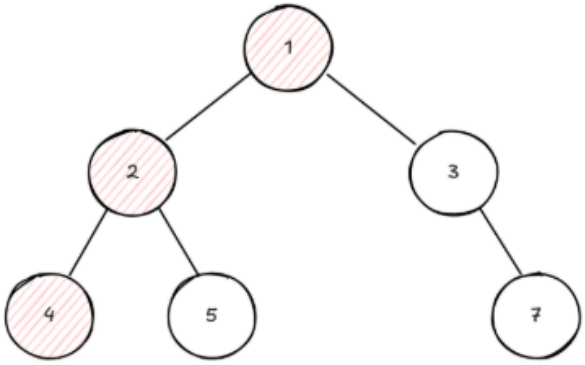
Metode Inorder Traversal adalah salah satu cara untuk menjelajahi Tree dengan urutan yang spesifik. Dalam metode ini, kita akan mengunjungi semua node yang ada di subTree kiri terlebih dahulu. Setelah selesai, kita pindah ke node akar dari Tree tersebut. Terakhir, kita mengunjungi semua node yang berada di subTree kanan. Berikut adalah source code C++ untuk inorder traversal.

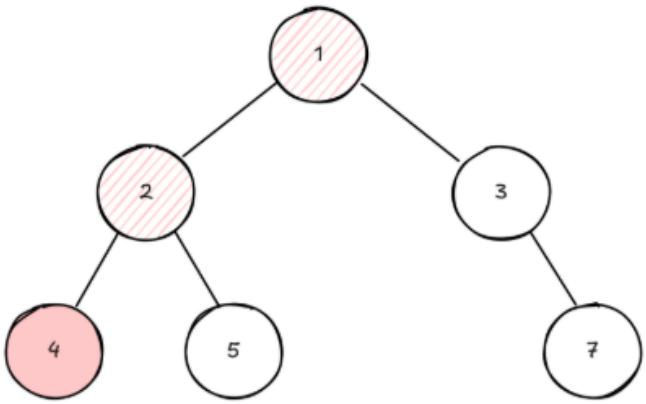
```
void inorderTraversal(Node* root) {
    if (root = nullptr) {
        return;
    }
    inorderTraversal(root >left); // Kunjungi subTree kiri
    cout < root >data < " "; // Kunjungi node akar
    inorderTraversal(root >right); // Kunjungi subTree kanan
}
```

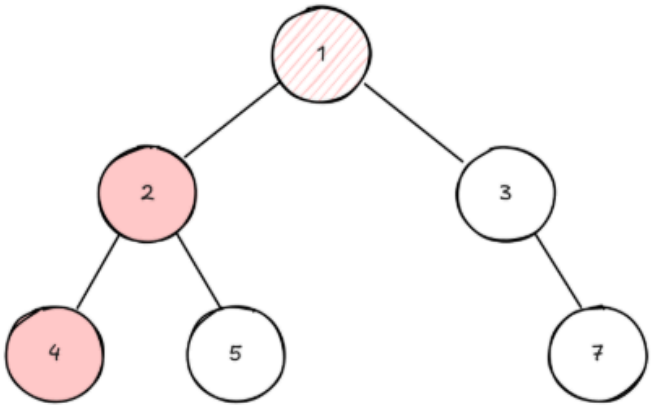
Berikut adalah contoh visualisasi dari urutan kunjungan inorder traversal.

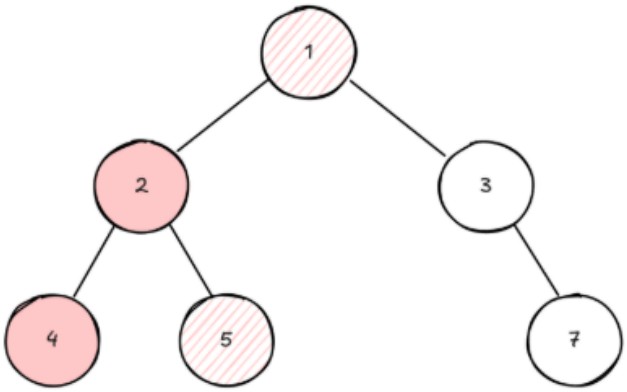


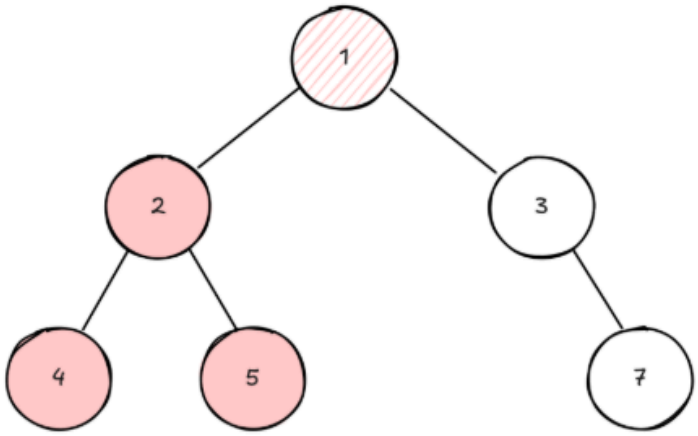

1
2
3
4
5
7

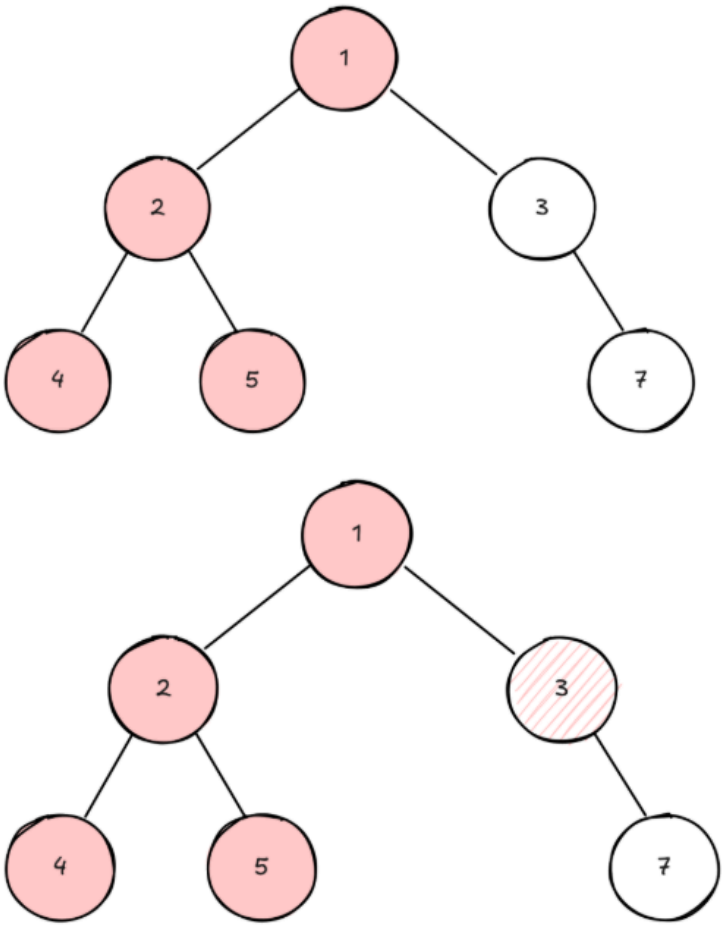

1
2
3
4
5
7

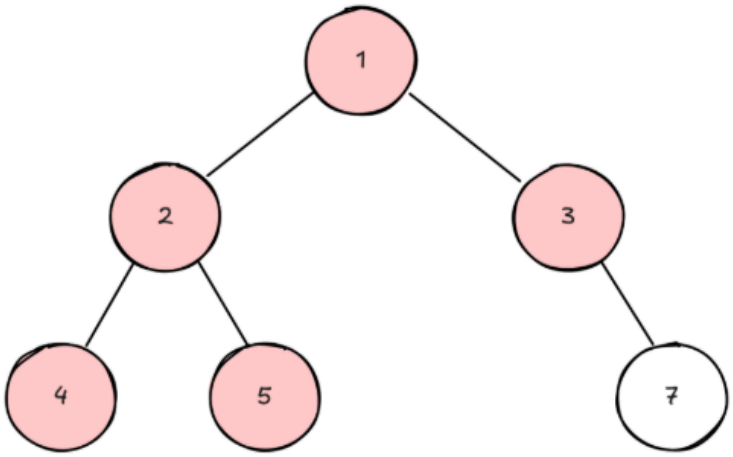

1
2
3
4
5
7

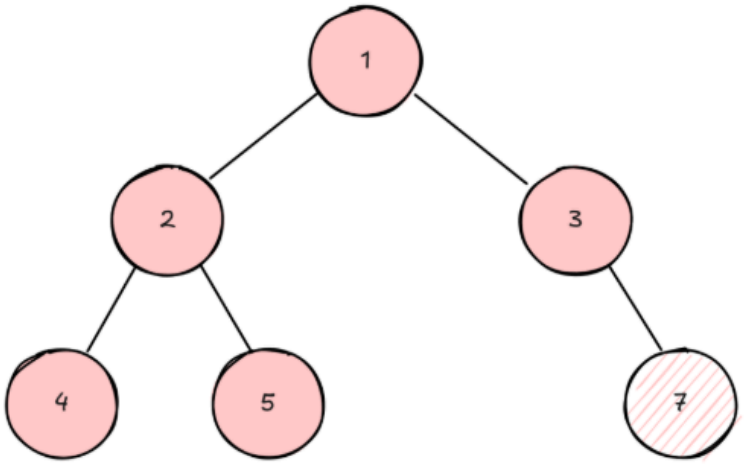

1
2
3
4
5
7

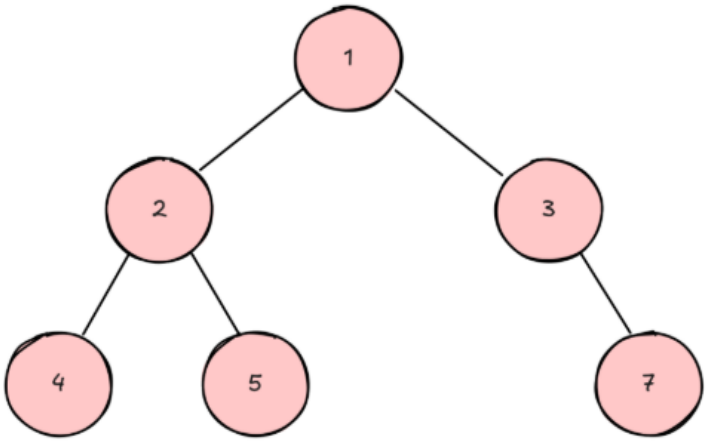

1
2
3
4
5
7


1
2
3
4
5
7


1
2
3
4
5
7
1
2
3
4
5
7


1
2
3
4
5
7


1
2
3
4
5
7


1
2
3
4
5
7

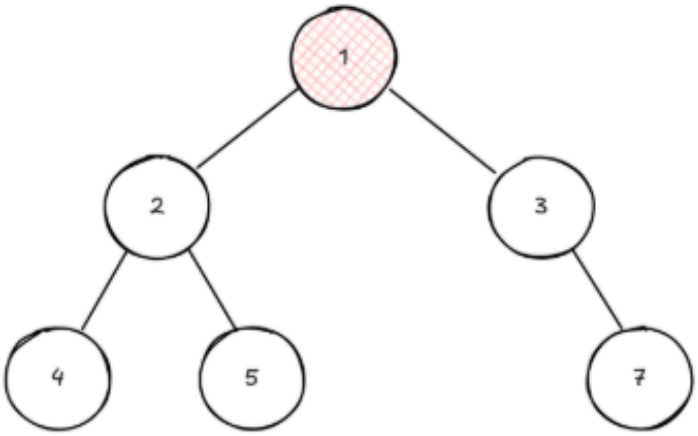
```
Output: 4,2,5,1,3,7
```

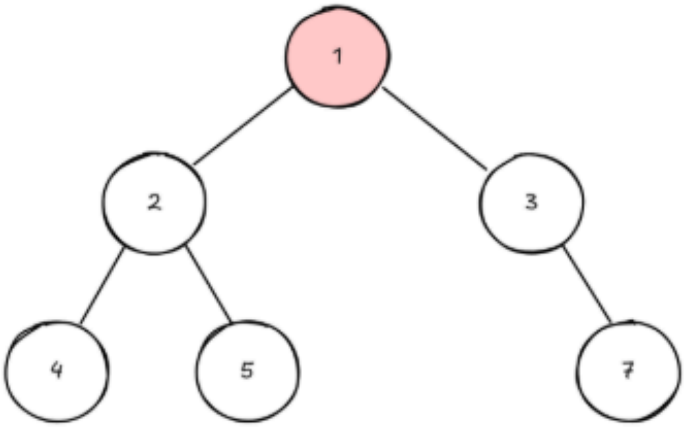
Pre order traversal

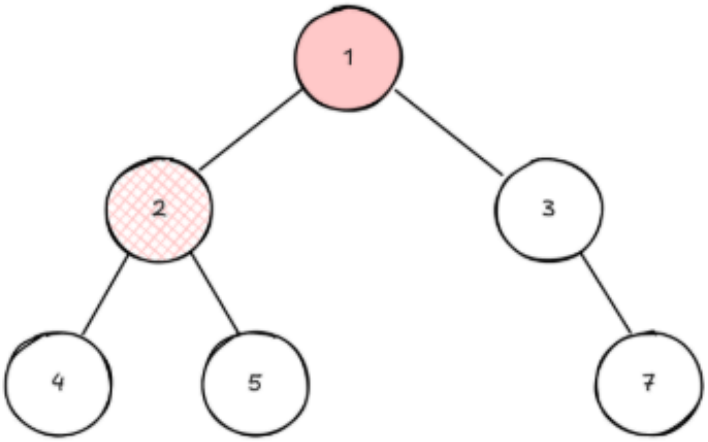
Metode Preorder Traversal adalah salah satu cara untuk menjelajahi Tree dengan urutan yang berbeda dari metode lainnya. Dalam metode ini, kita akan mengunjungi node akar terlebih dahulu. Setelah itu, kita akan beralih untuk mengunjungi semua node yang berada di subTree kiri. Terakhir, barulah kita mengunjungi semua node yang ada di subTree kanan. Berikut adalah source code C++ untuk preorder traversal.

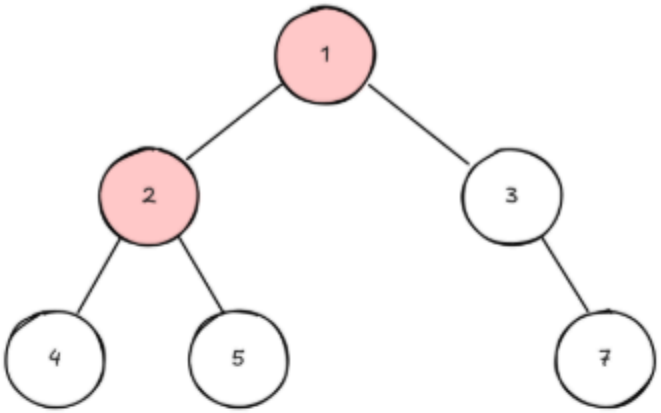
```
void preorderTraversal(Node* root) {
    if (root = nullptr) {
    return;
    }
    cout < root >data < " "; // Kunjungi node akar
    preorderTraversal(root >left); // Kunjungi subTree kiri
    preorderTraversal(root >right); // Kunjungi subTree kanan
}
```

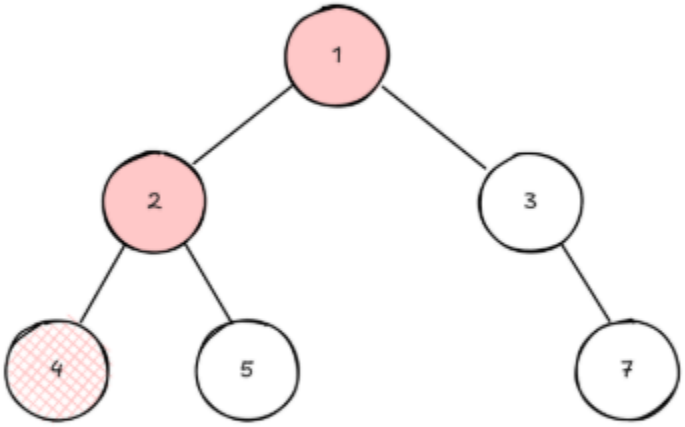
Berikut adalah contoh visualisasi dari urutan kunjungan preorder traversal.

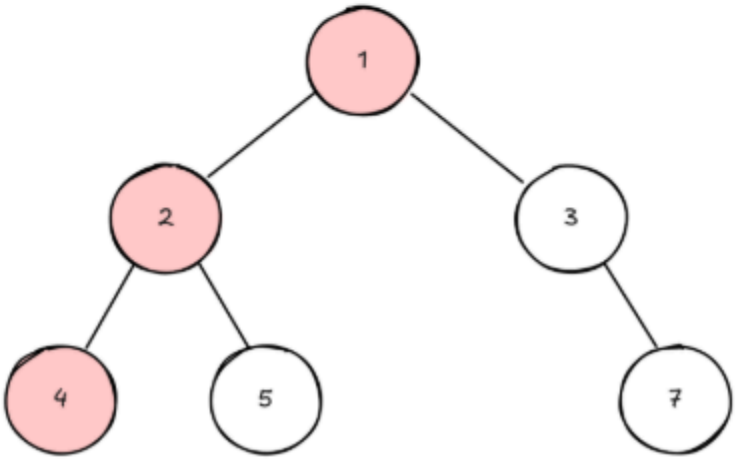

1
2
3
4
5
7

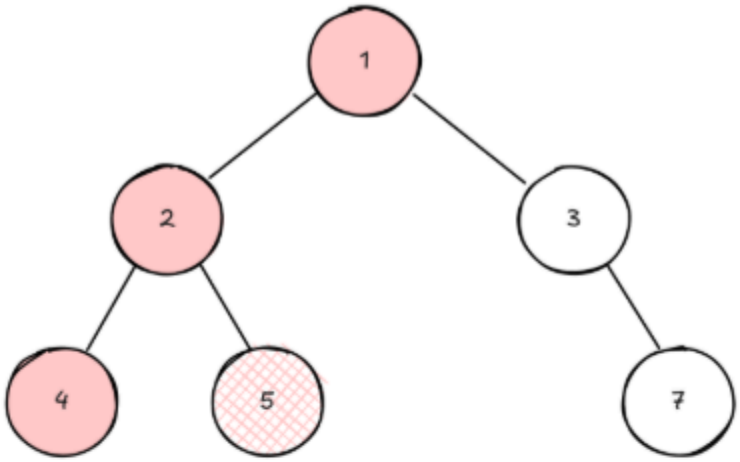

1
2
3
4
5
7

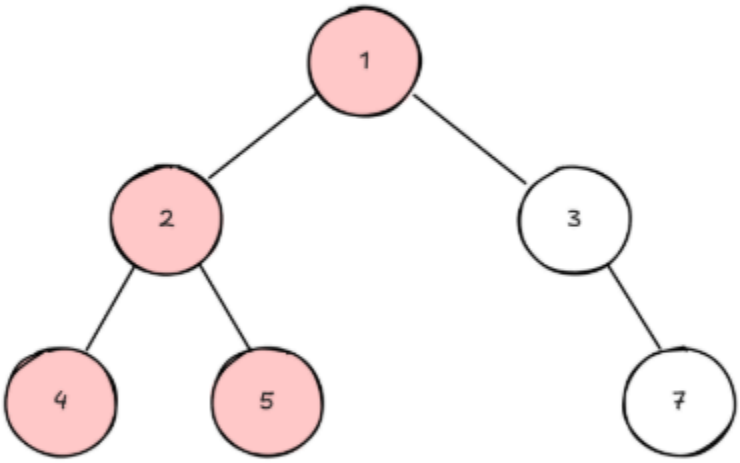

1
2
3
4
5
7

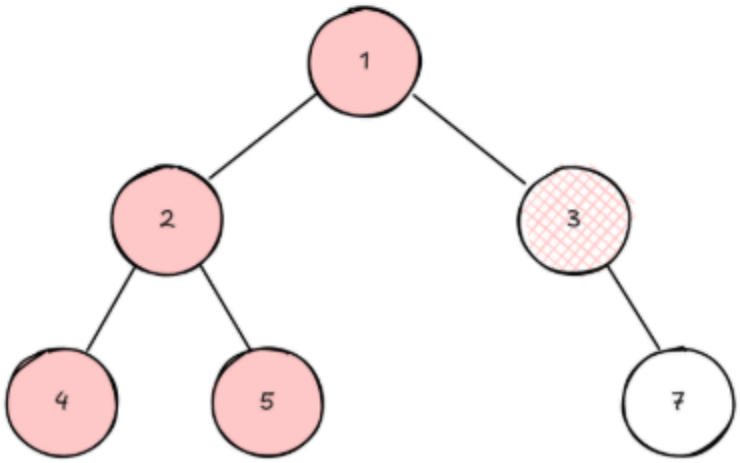

1
2
3
4
5
7

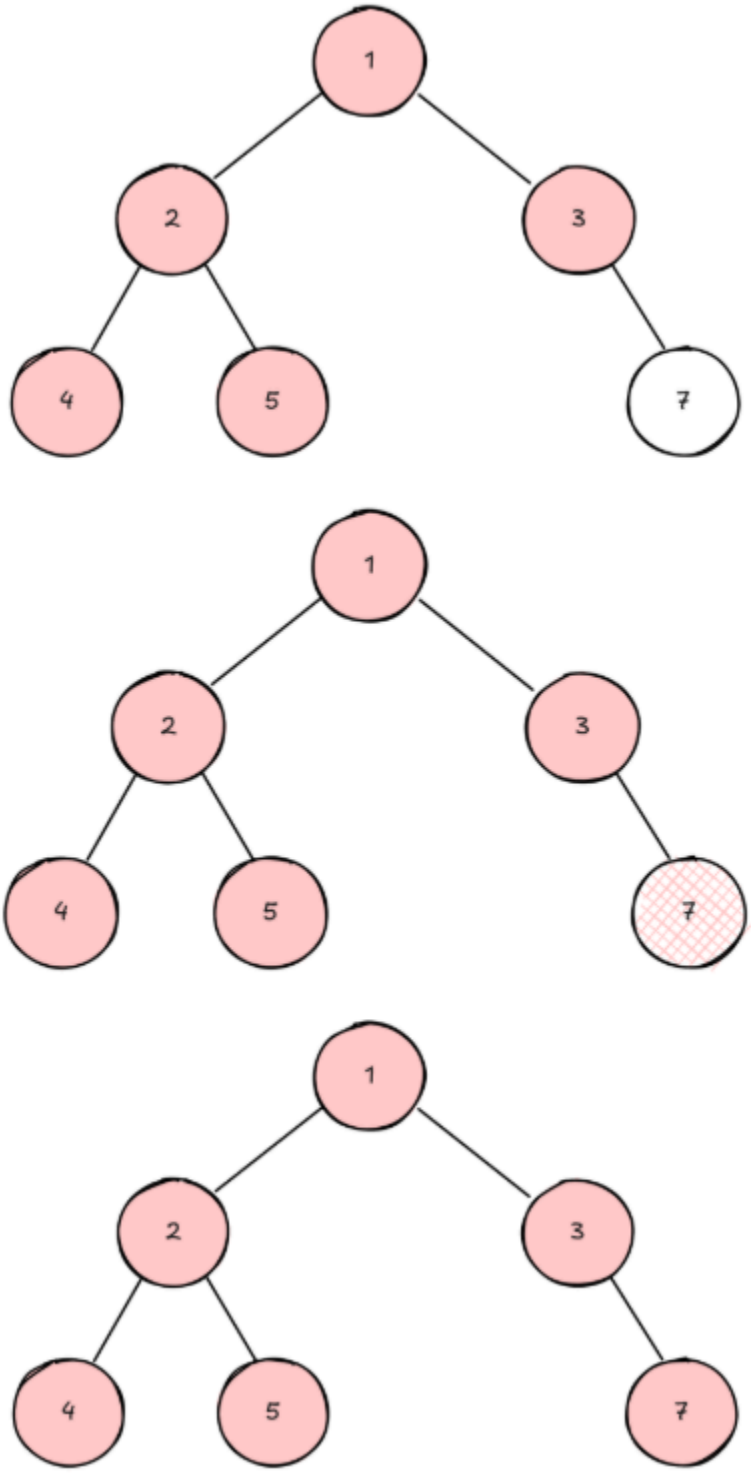

1
2
3
4
5
7


1
2
3
4
5
7


1
2
3
4
5
7


1
2
3
4
5
7


1
2
3
4
5
7


1
2
3
4
5
7
1
2
3
4
5
7
1
2
3
4
5
7

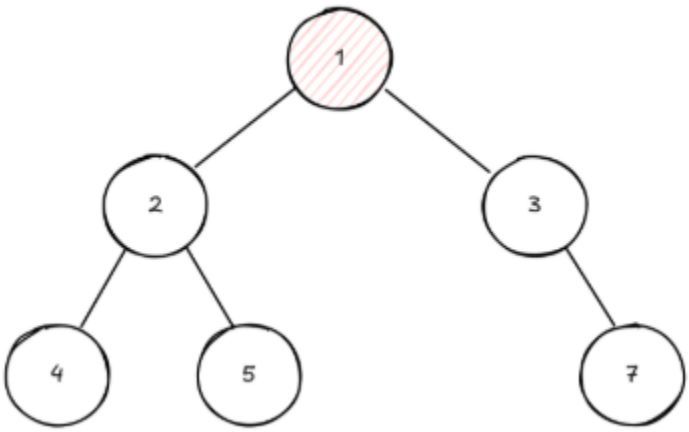
```
Output: 1,2,4,5,3,7
```

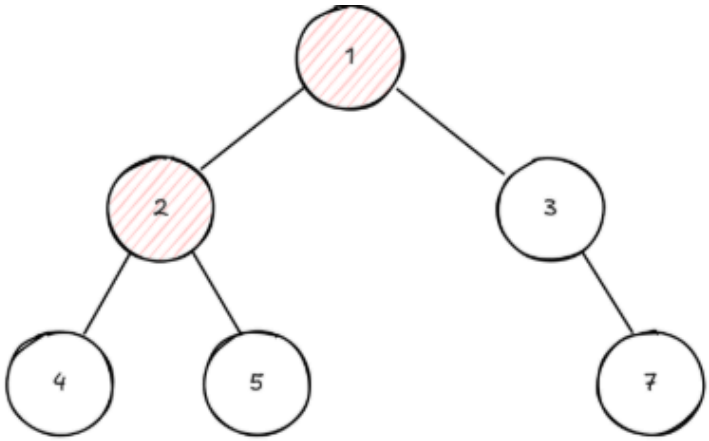
Post order traversal

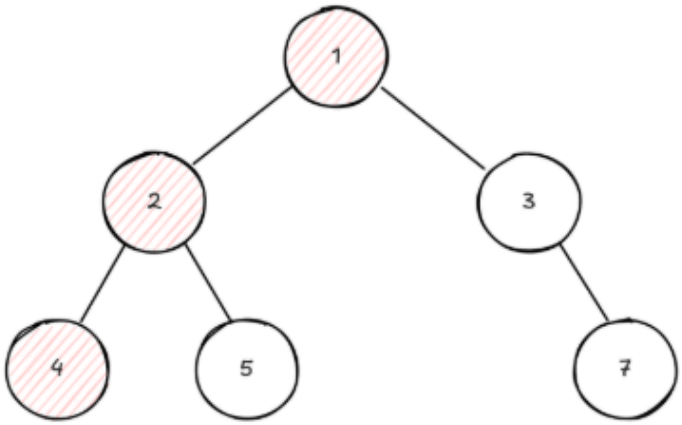
Metode Postorder Traversal adalah salah satu cara untuk menjelajahi Tree di mana kita mengunjungi node akar di akhir. Urutannya dimulai dengan mengunjungi semua node yang ada di subTree kiri. Setelah itu, kita berpindah untuk mengunjungi semua node yang berada di subTree kanan. Barulah setelah kedua subTree tersebut selesai dikunjungi, kita akan mengunjungi node akar.

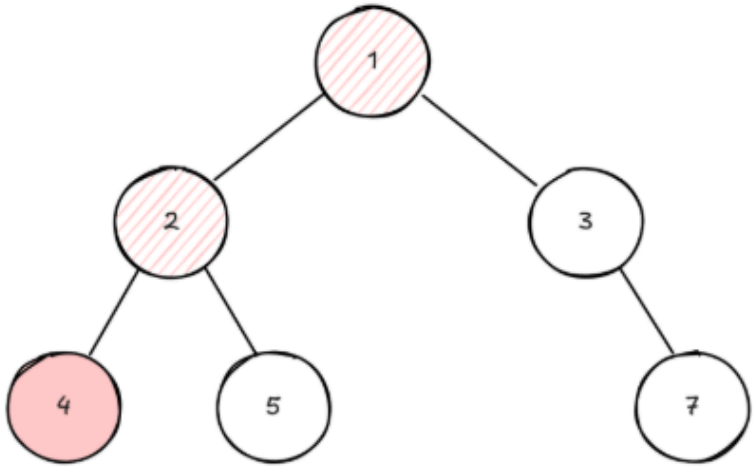
```
void postorderTraversal(Node* root) {
    if (root = nullptr) {
    return;
    }
    postorderTraversal(root >left); // Kunjungi subTree kiri
    postorderTraversal(root >right); // Kunjungi subTree kanan
    cout < root >data < " "; // Kunjungi node akar
}
```

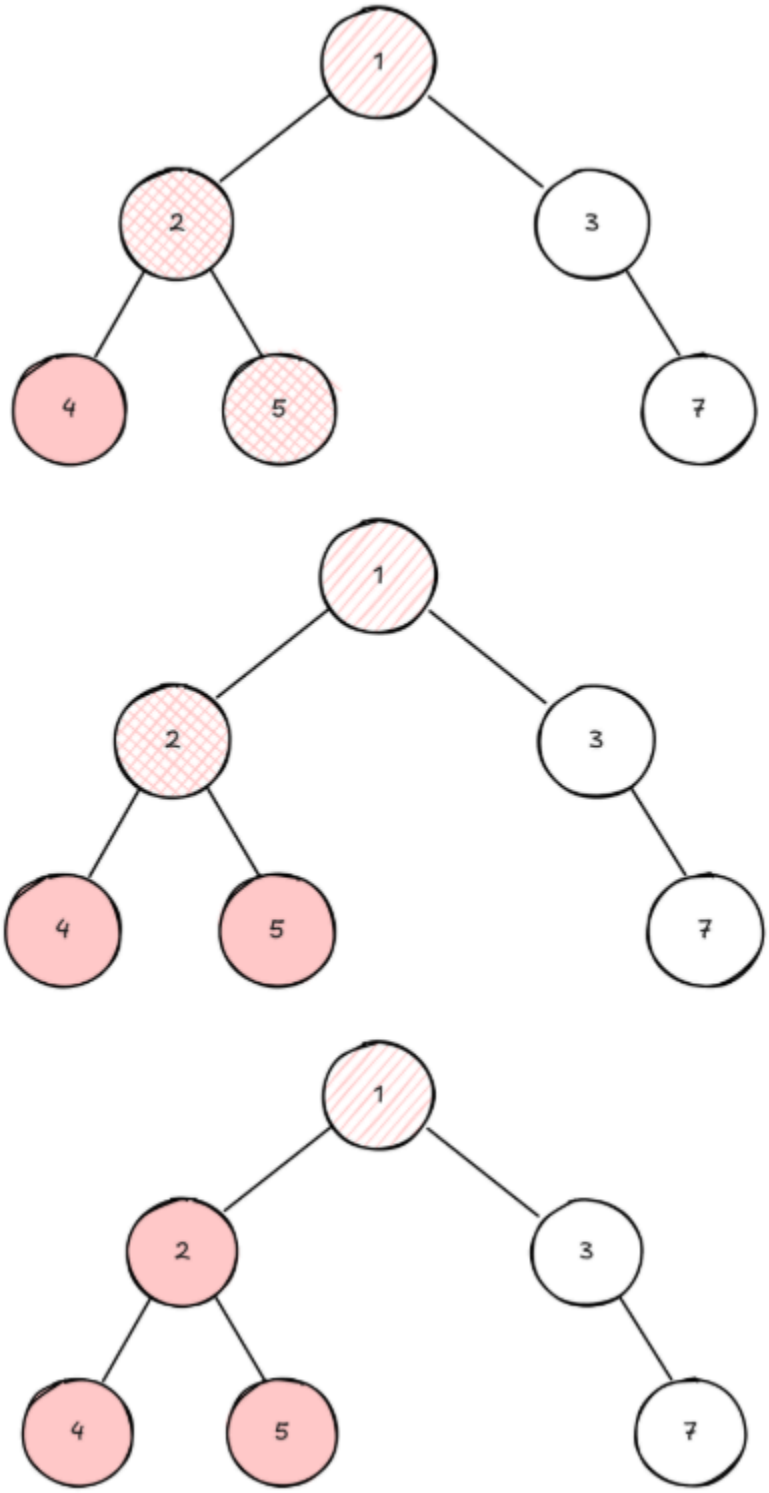
Berikut adalah contoh visualisasi dari urutan kunjungan Post-Order traversal.

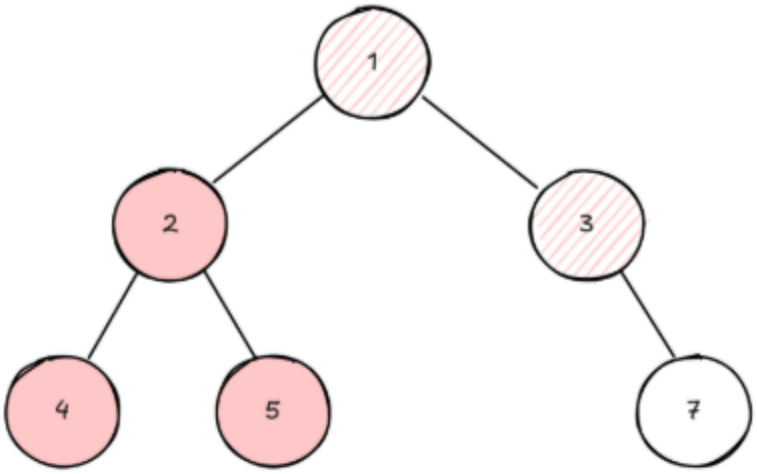


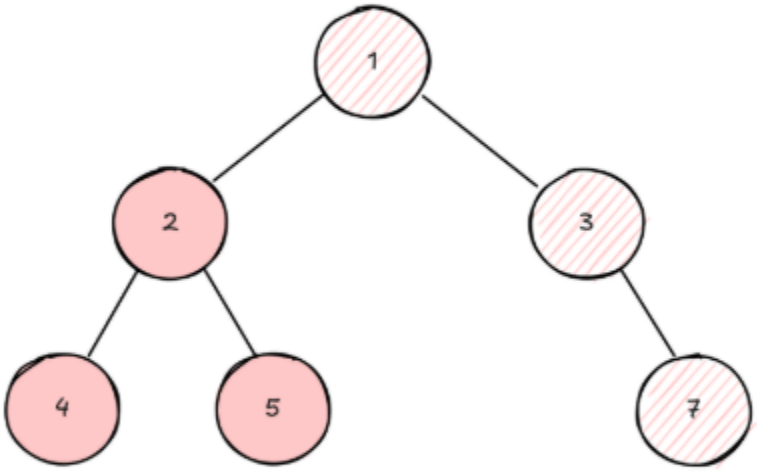

1
2
3
4
5
7

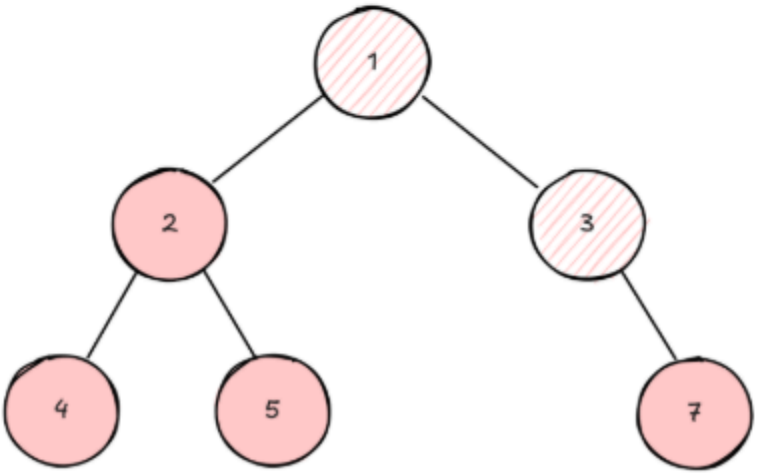

1
2
3
4
5
7

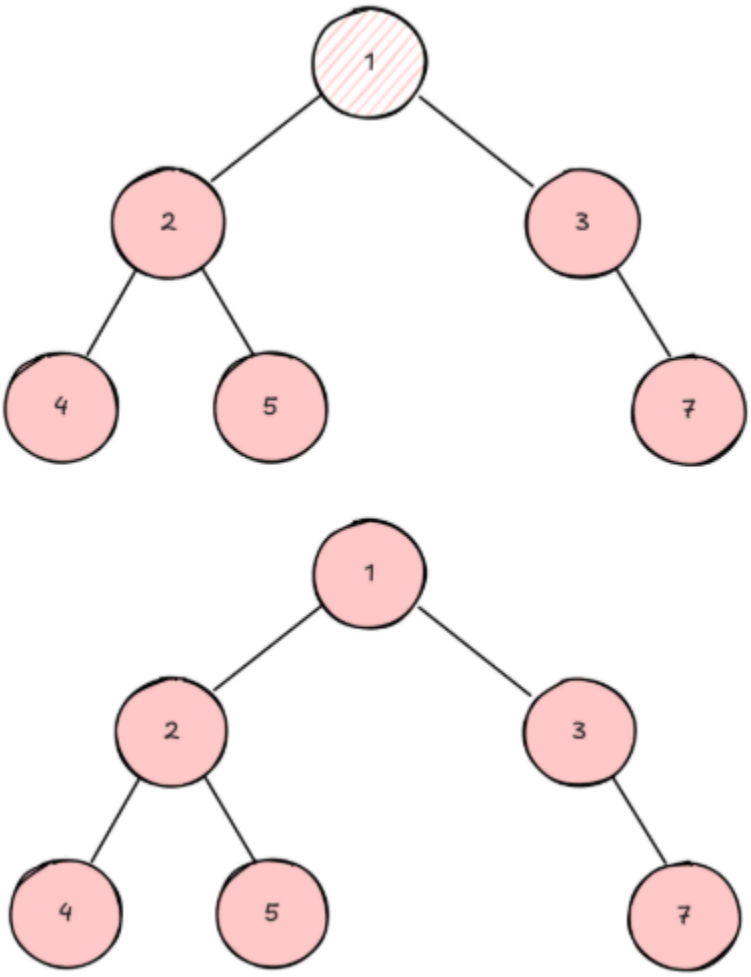

1
2
3
4
5
7


1
2
3
4
5
7
1
2
3
4
5
7
1
2
3
4
5
7


1
2
3
4
5
7


1
2
3
4
5
7


1
2
3
4
5
7




```
Output: 4,5,2,7,3,1
```

Breadth-First Search (BFS)

Algoritma BFS atau Level Order Traversal menelusuri Tree berdasarkan level, mulai dari root, kemudian semua node di level 1, lalu semua node di level 2, dan seterusnya. Untuk implementasinya,

```
#include <iostream>
#include <queue> // Library untuk menggunakan queue
using namespace std;

// ... (struktur Node yang sudah ada di atas)
// Fungsi untuk Level Order Traversal (BFS)

void bfs(Node* root) {
    if (root = nullptr) {
       return;
    }

    queue<Node > q; // Buat sebuah queue
    q.push(root); // Masukkan root ke dalam queue

    while (!q.empty()) {
        Node* current = q.front();
        q.pop();
        cout < current >data < " ";
        // Masukkan anak kiri jika ada
        if (current >left = nullptr) {
            q.push(current >left);
        }
        // Masukkan anak kanan jika ada
        if (current >right = nullptr) {
            q.push(current >right);
        }
}
```

Depth-First Search (DFS)

Algoritma DFS menelusuri Tree sejauh mungkin di sepanjang satu cabang sebelum kembali (backtrack). Tiga metode traversal yang sudah dibahas sebelumnya (In-Order, Pre-Order, Post-Order) sebenarnya adalah varian dari DFS. Implementasi ini menggunakan rekursi, yang memanfaatkan call stack internal dari program. Berikut adalah source code C++ untuk dfs.

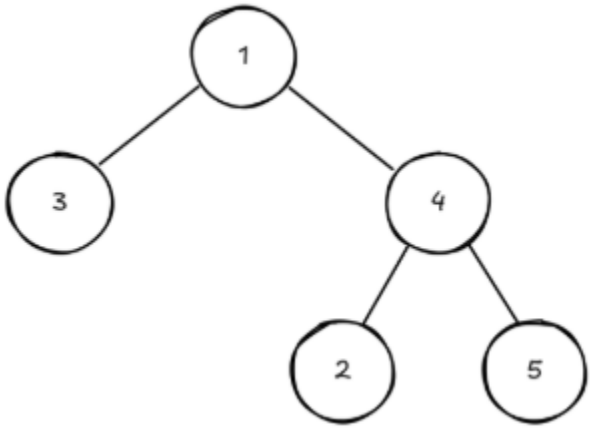
```
#include <iostream>
#include <stack> // Library untuk menggunakan stack
using namespace std;

//... (struktur Node yang sudah ada di atas)
// Fungsi untuk Pre-Order Traversal (DFS iteratif)

void dfs_iterative(Node* root) {
    if (root = nullptr) {
        return;
    }
    stack<Node > s; // Buat sebuah stack
    s.push(root); // Masukkan root ke dalam stack

    while (!s.empty()) {
    Node* current = s.top();
    s.pop();
    cout < current >data < " ";
    // Masukkan anak kanan lebih dulu agar anak kiri diproses lebih awal
    if (current >right = nullptr) {
        s.push(current >right);
    }
    if (current >left = nullptr) {
        s.push(current >left);
    }

    }
}
```

3. Binary Trees

Sebuah Tree disebut binary Tree jika setiap node memiliki nol, satu, atau dua anak maksimal 2 anak). Tree yang kosong juga merupakan binary Tree yang valid. Kita dapat membayangkan sebuah binary Tree terdiri dari satu root dan dua binary Tree yang terpisah, yang disebut subTree kiri dan kanan dari root.

Full binary tree

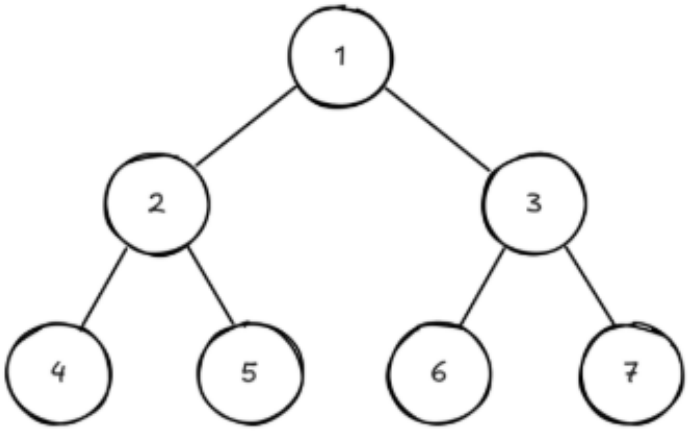
Full Binary Tree adalah jenis Tree di mana setiap node parent atau internal node memiliki tepat dua node anak atau tidak memiliki node anak sama sekali. Full Binary Tree juga dikenal sebagai proper binary Tree.



Perfect binary tree

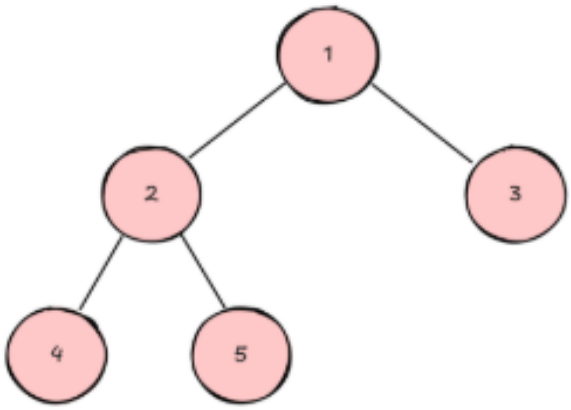
Perfect Binary Tree adalah jenis Tree di mana setiap node internal memiliki tepat dua node anak, dan semua node daun berada pada level yang sama. Ciri-ciri:

- Setiap node internal memiliki derajat (degree) 2, yang berarti memiliki 2 cabang.
- Sebuah Tree yang hanya terdiri dari satu node (tanpa anak) juga disebut Perfect Binary Tree dengan tinggi h = 0.
- Jika sebuah node memiliki tinggi h > 0, maka Tree tersebut dianggap Perfect Binary Tree jika kedua subTree-nya memiliki tinggi h-1 dan tidak saling tumpang tindih.



Complete Binary Tree

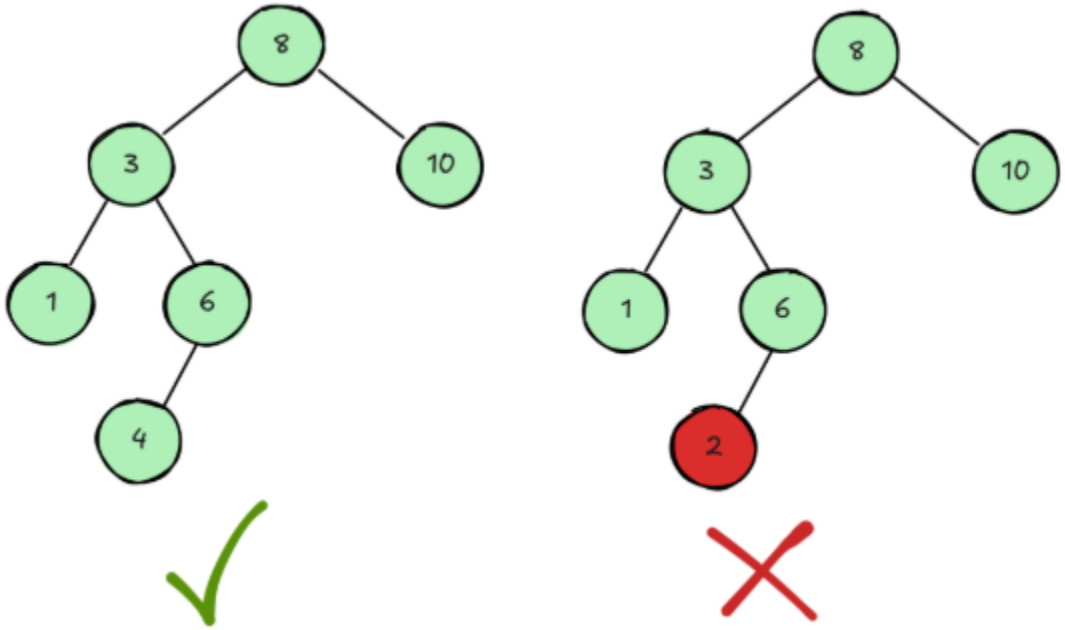
Complete Binary Tree adalah jenis Tree di mana semua level terisi penuh, kecuali kemungkinan level yang paling bawah. Pada level terendah ini, node diisi dari kiri ke kanan. Complete Binary Tree memiliki kemiripan dengan Full Binary Tree, namun perbedaannya terletak pada aturan pengisiannya: semua node daun (leaf nodes) harus mengisi dari sisi kiri terlebih dahulu. Ini berarti node daun terakhir mungkin tidak memiliki saudara (sibling) di sebelah kanannya. Dengan kata lain, sebuah Complete Binary Tree tidak selalu merupakan Full Binary Tree.



Binary Search Tree

Binary Search Tree adalah struktur data yang memungkinkan kita untuk mengelola daftar angka yang terurut secara efisien. Dalam sebuah BST, setiap node memiliki maksimal dua node anak, dan proses pencarian sebuah angka dapat dilakukan dengan kompleksitas waktu O(log(n)). Sebuah Binary Search Tree berbeda dari Binary Tree biasa karena memiliki aturan khusus:

- Semua node di subTree kiri nilainya lebih kecil dari node akar.
- Semua node di subTree kanan nilainya lebih besar dari node akar.
- Kedua subTree dari setiap node juga harus mengikuti aturan BST.



Sebagai contoh, Binary Tree yang ada pada gambar sebelah kanan bukan merupakan Binary Search Tree, karena pada subTree kanan dari node dengan nilai “3” terdapat nilai “2”, yang seharusnya lebih besar dari "3" agar memenuhi aturan BST.

Studi kasus

Seorang mahasiswa diberi dua hasil traversal dari sebuah binary tree:

- In order :D B E A F C
- Post order: D E B F C A

Tentukan pohon binernya!